

Bud — Reusable Companion Layer

scripts/bud-verify.ts

```
import { createSpark } from "../src/lib/spark";
import { createBud } from "../src/lib/bud";

async function main() {
  const spark = createSpark();
  await spark.initialize();

  spark.knowledge.addDocument({
    title: "Refund Policy",
    content: "Refunds are issued within 5 business days of a return request.",
    tags: ["policy", "refunds"],
  });

  const bud = createBud({ spark, personality: { name: "Bud", tone: "concise" } });

  const session = { sessionId: "s1", userId: "u1", product: "unibud" };

  console.log("--- Turn 1: simple question ---");
  const r1 = await bud.respond("What is your refund policy?", session);
  console.log(r1);

  console.log("\n--- Turn 2: multi-step message (should trigger planning) ---");
  const r2 = await bud.respond(
    "Check my refund status then email me a confirmation",
    session
  );
  console.log(r2);
  if (r2.trace.plannedTaskCount < 2) {
    throw new Error("Expected planIfNeeded to detect a multi-step request.");
  }

  console.log("\n--- Turn 3: memory should now include prior turns ---");
  const r3 = await bud.respond("What did I just ask you?", session);
  console.log(r3);
  if (r3.trace.memoryHits === 0) {
    throw new Error("Expected memory recall to find prior turns.");
  }
}
```

```

    console.log("\n--- Transcript (read back purely from Spark memory) ---");
    const transcript = bud.transcript(session.sessionId);
    console.log(JSON.stringify(transcript, null, 2));
    if (transcript.length < 2) {
        throw new Error("Expected at least two stored conversation turns.");
    }

    console.log("\n--- Spark health after Bud usage ---");
    console.log(spark.health());

    await spark.shutdown();
    console.log("\nBud verification completed successfully.");
}

main().catch((err) => {
    console.error("Bud verification failed:", err);
    throw err;
});

```

src/lib/bud/README.md

Bud — the reusable companion layer

Bud is a thin orchestration layer that sits between a user and Spark. It holds a personality and a conversation flow — nothing else. All memory, reasoning, planning, search, and text generation come from Spark.

```

```ts
import { createBud } from "@lib/bud";

const bud = createBud();

const response = await bud.respond("What's my next task, then remind me to email
 sessionId: "session_1",
 userId: "user_1",
 product: "unibud",
});

console.log(response.message);
console.log(response.trace); // { memoryHits, knowledgeHits, reasoningConfidence,

```

**Consumers should only ever** `import { createBud } from "@lib/bud"` . **Nothing under** `bud/orchestrator.ts` , `bud/actions/*` , `bud/adapters/*` , **etc. is a supported import path.**

## What Bud does NOT do

- Bud does not remember anything itself — `spark.memory` does.
- Bud does not search or retrieve anything itself — `spark.search` does.
- Bud does not reason about the message itself — `spark.reasoning` does.
- Bud does not plan multi-step goals itself — `spark.planning` does, once Bud has mechanically noticed the message looks like several steps.
- Bud does not call an AI provider, ever — `spark.writing` does, and only Spark decides which provider handles that call.

See `ARCHITECTURE.md` for the full request flow and the reasoning behind each of these boundaries.

## Sharing a Spark instance

By default `createBud()` creates its own private Spark instance. If you want multiple Bud instances (or Bud plus another future assistant) to share memory/knowledge/identity, pass one in:

```
import { createSpark } from "@lib/spark";
import { createBud } from "@lib/bud";

const spark = createSpark();
await spark.initialize();

const bud = createBud({ spark });
```

## Personality

Personality is plain configuration — a name, tone, and a few voice guidelines folded into the system prompt Bud sends to Spark's writing service. It contains no logic:

```
const bud = createBud({
 personality: {
 name: "Bud",
 tone: "concise",
 voice: ["Always suggest one concrete next step."],
 },
});
```

# Reusability

Bud has no reference to UNIBUD, My Realm, Orbit, or any other product. The `product` field on `BudSession` is passed straight through to Spark's identity/context services — Bud doesn't interpret it. Any product can use the same Bud module by supplying its own sessions.

```
`src/lib/bud/ARCHITECTURE.md`
```

```
```md
```

```
# Bud — Architecture
```

```
## Position in the ecosystem
```

User | ▼ Bud (personality + orchestration only) | ▼ Spark (all intelligence: memory, context, reasoning, planning, | knowledge, search, writing, translation, privacy, | security, automation, learning) ▼ AI Provider (Mock by default; OpenAI/Anthropic/Gemini/local when configured)

```
Bud never skips Spark to reach the AI provider layer directly, and Spark never imports or knows about Bud.
```

```
## Directory layout
```

`src/lib/bud/` |—— `index.ts` Public SDK — `createBud()`. The only supported import. |—— `config.ts` Static limits (recall/search result counts). |—— `types.ts` Session, message, and response shapes. |—— `personality.ts` Static tone/voice configuration. |—— `orchestrator.ts` The `respond()` flow, step by step. |—— `conversation.ts` Transcript formatting over Spark memory (no storage of its own). |—— `prompts/` |—— `systemPrompt.ts` Builds the system prompt from personality config. |—— `userPrompt.ts` Builds the user-turn prompt from Spark's outputs. |—— `actions/` |—— `recallMemory.ts` Calls `spark.memory.recall` |—— `searchKnowledge.ts` Calls `spark.search.search` |—— `reason.ts` Calls `spark.reasoning.analyze` |—— `planIfNeeded.ts` Mechanical step detection -> `spark.planning.createPlan` |—— `generateResponse.ts` Calls `spark.writing.draft` |—— `storeInteraction.ts` Calls `spark.memory.remember` |—— `context/` |—— `buildContext.ts` Ensures `spark.identity` has a context for the session |—— `adapters/` |—— `sparkPort.ts` The narrow interface Bud is allowed to depend on |—— `liveSparkAdapter.ts` Wraps a real Spark instance to

satisfy that interface

The respond() flow

`bud.respond(message, session)` runs exactly these steps, in order, every time:

1. **Build context** – `context/buildContext.ts` asks Spark's Identity service whether this session already has a context; creates one if not.
2. **Recall memory** – `actions/recallMemory.ts` asks Spark for prior episodic memory tied to this session.
3. **Search knowledge** – `actions/searchKnowledge.ts` asks Spark's Search service (which itself only talks to Knowledge) for anything relevant to the message.
4. **Reason** – `actions/reason.ts` hands the message plus everything recalled/found to Spark's Reasoning service and gets back an answer, a confidence score, and an explainable step trace.
5. **Plan (conditionally)** – `actions/planIfNeeded.ts` does a mechanical check (keyword/punctuation split, not semantic understanding) for whether the message looks like multiple sequenced asks. If so, it hands the extracted steps to Spark's Planning service. If not, no plan is created and no planning call is made.
6. **Generate response** – `actions/generateResponse.ts` builds a prompt (system prompt from personality + user prompt from Spark's reasoning/memory/knowledge outputs) and calls `spark.writing.draft()`. Spark resolves whichever `AIProvider` is registered – Bud never knows or cares which one.
7. **Store the turn** – `actions/storeInteraction.ts` writes both the user's message and Bud's reply back into Spark memory as episodic records, tagged `user`/`bud`.

Every one of those seven steps is a call through `BudSparkPort` – the orchestrator (`orchestrator.ts`) contains no branching logic that depends on the *content* of the message beyond step 5's mechanical split. It doesn't summarize, doesn't judge relevance, doesn't decide what's true – it delegates all of that.

Why `adapters/sparkPort.ts` exists

`BudSparkPort` is a deliberately narrow re-typing of Spark's public surface – only `identity`, `memory`, `search`, `reasoning`, `planning`, and `writing`. Two consequences:

- **It's structurally impossible for Bud's action files to reach a**

capability that isn't listed** – there is no `providers` field on the port, so Bud cannot call an `AIProvider` even by mistake.

- **Bud is independently testable.** Anything satisfying `BudSparkPort`'s shape can stand in for Spark in a test, without spinning up a real Spark instance. `adapters/liveSparkAdapter.ts` is the only file that touches the concrete `Spark` class.

Where conversation history actually lives

`conversation.ts` has no storage of its own. `getTranscript()` reads Spark's episodic memory for a session and reshapes matched user/bud record pairs into `ConversationTurn[]`. If Spark's memory for that session is cleared, the transcript is gone – there is no shadow copy inside Bud.

Product independence

`BudSession.product` is passed straight through to Spark's identity and context services without Bud inspecting or branching on its value. There is no `if (product === "unibud")` anywhere in this module, and there must never be – that would turn Bud from a reusable companion into a product-specific one. New products reuse this exact module by supplying their own `product` string.

What's deliberately not here yet

- No persistence beyond whatever Spark itself persists (currently in-memory, see the Spark scaffold's README).
- No streaming responses – `writing.draft()` returns a complete string.
- No multi-turn planning follow-through (Bud creates a plan via step 5 but does not yet check `spark.planning.nextTask()` on later turns to walk through it turn-by-turn).
- No integration into UNIBUD, My Realm, Orbit, or any application.

src/lib/bud/index.ts

```
/**
 * Public Bud SDK.
 *
 * Consumers should only ever import from here:
 *
 * import { createBud } from "@lib/bud";
 *
 * Never import from bud/orchestrator, bud/actions/*, bud/adapters/*,
```

```

* etc. directly - those are internal implementation detail.
*/

import { createSpark, type Spark } from "../spark";
import { createLiveSparkAdapter } from "../adapters/liveSparkAdapter";
import type { BudSparkPort } from "../adapters/sparkPort";
import { createPersonality, type BudPersonality } from "../personality";
import { resolveLimits, type BudLimits } from "../config";
import { orchestrateRespond } from "../orchestrator";
import { getTranscript } from "../conversation";
import type { BudResponse, BudSession, ConversationTurn } from "../types";

export interface BudConfig {
  /** Provide an existing Spark instance to share it across products; otherwise B
  spark?: Spark;
  personality?: Partial<BudPersonality>;
  limits?: Partial<BudLimits>;
}

export interface Bud {
  respond(message: string, session: BudSession): Promise<BudResponse>;
  transcript(sessionId: string, limit?: number): ConversationTurn[];
  readonly personality: BudPersonality;
}

export function createBud(config: BudConfig = {}): Bud {
  const spark = config.spark ?? createSpark();
  const sparkPort: BudSparkPort = createLiveSparkAdapter(spark);
  const personality = createPersonality(config.personality);
  const limits = resolveLimits(config.limits);

  return {
    personality,

    async respond(message: string, session: BudSession): Promise<BudResponse> {
      return orchestrateRespond(sparkPort, personality, limits, message, session)
    },

    transcript(sessionId: string, limit?: number): ConversationTurn[] {
      return getTranscript(sparkPort, sessionId, limit);
    },
  };
}

export type { BudSession, BudResponse, BudMessage, ConversationTurn } from "../typ
export type { BudPersonality } from "../personality";
export type { BudLimits } from "../config";

```

src/lib/bud/types.ts

```
/**
 * Bud never defines its own memory, reasoning, or knowledge types –
 * those belong to Spark. This file only defines the shapes Bud needs
 * for orchestration: sessions, messages, and the final response.
 */

export interface BudSession {
  sessionId: string;
  userId: string;
  /** The product Bud is embedded in (e.g. "unibud", "campus"). Passed through to
   product: string;
   locale?: string;
   timezone?: string;
 }

export interface BudMessage {
  role: "user" | "bud";
  content: string;
  timestamp: string;
 }

export interface BudResponse {
  message: string;
  sessionId: string;
  /** Surfaced for debugging/observability – never re-derived by Bud itself. */
  trace: {
    memoryHits: number;
    knowledgeHits: number;
    reasoningConfidence: number;
    plannedTaskCount: number;
    provider: string;
  };
 }

export interface ConversationTurn {
  user: BudMessage;
  bud: BudMessage;
 }
```

src/lib/bud/config.ts

```
/**
 * Static configuration only. No behavior lives here – these are just
 * the knobs the orchestrator reads.
 */

export interface BudLimits {
  /** Max prior memory records recalled per turn. */
  memoryRecallLimit: number;
  /** Max knowledge/search results considered per turn. */
  knowledgeSearchLimit: number;
}

export const DEFAULT_BUD_LIMITS: BudLimits = {
  memoryRecallLimit: 10,
  knowledgeSearchLimit: 5,
};

export interface BudConfigOptions {
  name?: string;
  limits?: Partial<BudLimits>;
}

export function resolveLimits(overrides?: Partial<BudLimits>): BudLimits {
  return { ...DEFAULT_BUD_LIMITS, ...overrides };
}
```

src/lib/bud/personality.ts

```
/**
 * Personality is pure configuration: a name, a tone, and a short
 * description used to build prompts. It contains no logic and makes
 * no decisions – it's data that prompts/systemPrompt.ts reads.
 */

import type { WritingTone } from "../../spark/intelligence/writing/interface";

export interface BudPersonality {
  name: string;
  description: string;
  tone: WritingTone;
  /** Short first-person guidance folded into the system prompt. */
}
```

```

    voice: string[];
}

export const DEFAULT_BUD_PERSONALITY: BudPersonality = {
  name: "Bud",
  description: "A supportive, plain-spoken companion.",
  tone: "casual",
  voice: [
    "Keep answers short and easy to act on.",
    "Be encouraging without being saccharine.",
    "Say plainly when you don't know something.",
  ],
};

export function createPersonality(
  overrides?: Partial<BudPersonality>
): BudPersonality {
  return { ...DEFAULT_BUD_PERSONALITY, ...overrides };
}

```

src/lib/bud/orchestrator.ts

```

import type { BudSparkPort } from "../adapters/sparkPort";
import type { BudPersonality } from "../personality";
import type { BudLimits } from "../config";
import type { BudResponse, BudSession } from "../types";
import { buildContext } from "../context/buildContext";
import { recallMemory } from "../actions/recallMemory";
import { searchKnowledge } from "../actions/searchKnowledge";
import { reason } from "../actions/reason";
import { planIfNeeded } from "../actions/planIfNeeded";
import { generateResponse } from "../actions/generateResponse";
import { storeInteraction } from "../actions/storeInteraction";

/**
 * The entire orchestration Bud performs, in order:
 *
 * 1. Build conversation context (ensure Spark identity exists)
 * 2. Recall memory from Spark
 * 3. Search Spark's knowledge
 * 4. Ask Spark to reason over message + recalled facts
 * 5. Mechanically detect multi-step requests and hand planning to Spark
 * 6. Ask Spark to draft the final response text
 * 7. Store the turn back through Spark memory

```

```

*
* Every step above calls into `spark` (a BudSparkPort) and nothing else.
* There is no branch here that reasons about content, generates text
* itself, or calls a provider – that's the point.
*/
export async function orchestrateRespond(
  spark: BudSparkPort,
  personality: BudPersonality,
  limits: BudLimits,
  message: string,
  session: BudSession
): Promise<BudResponse> {
  const context = buildContext(spark, session);

  const memoryRecords = recallMemory(
    spark,
    context.sessionId,
    limits.memoryRecallLimit
  );

  const knowledgeResults = await searchKnowledge(
    spark,
    message,
    limits.knowledgeSearchLimit
  );

  const reasoning = await reason(spark, message, memoryRecords, knowledgeResults)

  const plan = planIfNeeded(spark, message);

  const writing = await generateResponse(spark, personality, {
    message,
    memoryRecords,
    knowledgeResults,
    reasoning,
  });

  storeInteraction(spark, {
    sessionId: context.sessionId,
    userId: context.userId,
    userMessage: message,
    budMessage: writing.text,
  });

  return {
    message: writing.text,
    sessionId: context.sessionId,
  }
}

```

```

    trace: {
      memoryHits: memoryRecords.length,
      knowledgeHits: knowledgeResults.length,
      reasoningConfidence: reasoning.confidence,
      plannedTaskCount: plan?.tasks.length ?? 0,
      provider: writing.provider,
    },
  };
}

```

src/lib/bud/conversation.ts

```

import type { BudSparkPort } from "../adapters/sparkPort";
import type { MemoryRecord } from "../spark/memory/interface";
import type { ConversationTurn, BudMessage } from "../types";

/**
 * Bud keeps no conversation store of its own. This module only reads
 * back what Spark already has in episodic memory and reshapes it into
 * a transcript. If Spark's memory is cleared, the transcript is gone -
 * there is no separate copy living in Bud.
 */
export function getTranscript(
  spark: BudSparkPort,
  sessionId: string,
  limit = 50
): ConversationTurn[] {
  const records = spark.memory
    .recall({ sessionId, kind: "episodic", limit: limit * 2 })
    .slice()
    .reverse(); // recall() returns newest-first; transcripts read oldest-first

  const turns: ConversationTurn[] = [];
  let pendingUser: BudMessage | null = null;

  for (const record of records) {
    const message = toMessage(record);
    if (message.role === "user") {
      pendingUser = message;
    } else if (pendingUser) {
      turns.push({ user: pendingUser, bud: message });
      pendingUser = null;
    }
  }
}

```

```

    return turns;
}

function toMessage(record: MemoryRecord): BudMessage {
    const isUser = record.tags?.includes("user");
    const content = record.content.replace(/^(User|Bud):\s*/, "");
    return {
        role: isUser ? "user" : "bud",
        content,
        timestamp: record.createdAt,
    };
}

```

src/lib/bud/context/buildContext.ts

```

import type { BudSparkPort } from "../../adapters/sparkPort";
import type { BudSession } from "../../types";

export interface ConversationContext {
    sessionId: string;
    userId: string;
    product: string;
}

/**
 * Ensures Spark has an identity context for this session (creating one
 * on first contact) and returns the minimal context the rest of the
 * orchestration needs. Bud does not track identity itself – Spark owns
 * that state.
 */
export function buildContext(
    spark: BudSparkPort,
    session: BudSession
): ConversationContext {
    const existing = spark.identity.getContext(session.sessionId);

    if (!existing) {
        spark.identity.createContext({
            userId: session.userId,
            sessionId: session.sessionId,
            product: session.product,
            locale: session.locale,
            timezone: session.timezone,

```

```

    });
}

return {
  sessionId: session.sessionId,
  userId: session.userId,
  product: session.product,
};
}

```

src/lib/bud/adapters/sparkPort.ts

```

/**
 * BudSparkPort is the ONLY thing Bud's orchestration code depends on —
 * never the concrete Spark class directly. This keeps Bud honest about
 * "orchestration only": if a capability isn't listed here, Bud cannot
 * reach for it, including AI providers, which are deliberately absent
 * from this port.
 *
 * It is a subset of Spark's public API, re-typed narrowly so Bud's
 * intent (recall memory, search knowledge, reason, plan, draft text,
 * store memory) is visible at the type level.
 */

import type { IdentityContext } from "../../spark/core/identity/interface";
import type { ReasoningResult } from "../../spark/core/reasoning/interface";
import type { Plan } from "../../spark/core/planning/interface";
import type { MemoryKind, MemoryRecord } from "../../spark/memory/interface";
import type { SearchResult } from "../../spark/intelligence/search/interface";
import type { WritingResult, WritingTone } from "../../spark/intelligence/writing";

export interface BudSparkPort {
  identity: {
    createContext(input: {
      userId: string;
      sessionId: string;
      product: string;
      locale?: string;
      timezone?: string;
    }): IdentityContext;
    getContext(sessionId: string): IdentityContext | undefined;
  };

  memory: {

```

```

    remember(input: {
      kind: MemoryKind;
      content: string;
      sessionId?: string;
      userId?: string;
      tags?: string[];
    }): MemoryRecord;
    recall(query: {
      sessionId?: string;
      userId?: string;
      kind?: MemoryKind;
      limit?: number;
    }): MemoryRecord[];
  };

  search: {
    search(query: string, limit?: number): Promise<SearchResult[]>;
  };

  reasoning: {
    analyze(input: {
      question: string;
      facts?: string[];
    }): Promise<ReasoningResult>;
  };

  planning: {
    createPlan(goal: string, taskTitles: string[]): Plan;
  };

  writing: {
    draft(input: {
      prompt: string;
      tone?: WritingTone;
      maxLength?: number;
    }): Promise<WritingResult>;
  };
}

```

src/lib/bud/adapters/liveSparkAdapter.ts

```

import type { Spark } from "../../spark";
import type { BudSparkPort } from "../sparkPort";

```

```

/**
 * Wraps a concrete Spark instance so Bud's orchestrator only ever sees
 * the narrow BudSparkPort surface. Swapping this file for a test double
 * (e.g. an in-memory fake) is the only thing needed to unit-test Bud
 * without a real Spark instance.
 */
export function createLiveSparkAdapter(spark: Spark): BudSparkPort {
  return {
    identity: {
      createContext: (input) => spark.identity.createContext(input),
      getContext: (sessionId) => spark.identity.getContext(sessionId),
    },
    memory: {
      remember: (input) => spark.memory.remember(input),
      recall: (query) => spark.memory.recall(query),
    },
    search: {
      search: (query, limit) => spark.search.search(query, limit),
    },
    reasoning: {
      analyze: (input) => spark.reasoning.analyze(input),
    },
    planning: {
      createPlan: (goal, taskTitles) => spark.planning.createPlan(goal, taskTitle
    },
    writing: {
      draft: (input) => spark.writing.draft(input),
    },
  };
}

```

src/lib/bud/actions/recallMemory.ts

```

import type { BudSparkPort } from "../../adapters/sparkPort";
import type { MemoryRecord } from "../../spark/memory/interface";

/**
 * Thin wrapper around Spark's memory recall. Bud does not implement
 * any memory logic itself – it only asks Spark for what it remembers.
 */
export function recallMemory(
  spark: BudSparkPort,
  sessionId: string,
  limit: number

```

```

): MemoryRecord[] {
  return spark.memory.recall({ sessionId, limit });
}

```

src/lib/bud/actions/searchKnowledge.ts

```

import type { BudSparkPort } from "../../adapters/sparkPort";
import type { SearchResult } from "../../spark/intelligence/search/interface";

/**
 * Thin wrapper around Spark's search service. Bud never implements its
 * own search or retrieval – it only asks Spark.
 */
export async function searchKnowledge(
  spark: BudSparkPort,
  query: string,
  limit: number
): Promise<SearchResult[]> {
  return spark.search.search(query, limit);
}

```

src/lib/bud/actions/reason.ts

```

import type { BudSparkPort } from "../../adapters/sparkPort";
import type { ReasoningResult } from "../../spark/core/reasoning/interface";
import type { MemoryRecord } from "../../spark/memory/interface";
import type { SearchResult } from "../../spark/intelligence/search/interface";

/**
 * Assembles the facts Spark's reasoning service should consider – from
 * recalled memory and search results – and delegates the actual
 * analysis to Spark. Bud does not reason on its own.
 */
export async function reason(
  spark: BudSparkPort,
  message: string,
  memoryRecords: MemoryRecord[],
  knowledgeResults: SearchResult[]
): Promise<ReasoningResult> {
  const facts = [
    ...memoryRecords.map((m) => m.content),

```

```

    ...knowledgeResults.map((k) => `${k.title}: ${k.snippet}`),
  ];

  return spark.reasoning.analyze({ question: message, facts });
}

```

src/lib/bud/actions/planIfNeeded.ts

```

import type { BudSparkPort } from "../../adapters/sparkPort";
import type { Plan } from "../../spark/core/planning/interface";

const STEP_SEPARATORS = /\bthen\b|;\n-|\n\d+\./i;

/**
 * This is mechanical text splitting, not reasoning about intent – Bud
 * detects a shape (multiple steps separated by "then"/semicolons/list
 * markers) and hands the actual planning off to Spark. If the message
 * doesn't look like a multi-step request, no plan is created.
 */
export function planIfNeeded(spark: BudSparkPort, message: string): Plan | null {
  if (!STEP_SEPARATORS.test(message)) {
    return null;
  }

  const steps = message
    .split(STEP_SEPARATORS)
    .map((s) => s.replace(/^[s\-\.\d]+/, "").trim())
    .filter((s) => s.length > 0);

  if (steps.length < 2) {
    return null;
  }

  return spark.planning.createPlan(message, steps);
}

```

src/lib/bud/actions/generateResponse.ts

```

import type { BudSparkPort } from "../../adapters/sparkPort";
import type { BudPersonality } from "../../personality";
import type { ReasoningResult } from "../../spark/core/reasoning/interface";

```

```

import type { MemoryRecord } from "../../spark/memory/interface";
import type { SearchResult } from "../../spark/intelligence/search/interface";
import { buildSystemPrompt } from "../../prompts/systemPrompt";
import { buildUserPrompt } from "../../prompts/userPrompt";
import type { WritingResult } from "../../spark/intelligence/writing/interface";

/**
 * Delegates text generation to Spark.writing (which in turn delegates
 * to whatever AIProvider is configured). Bud never calls a provider
 * directly – this is the only path to generated text.
 */
export async function generateResponse(
  spark: BudSparkPort,
  personality: BudPersonality,
  input: {
    message: string;
    memoryRecords: MemoryRecord[];
    knowledgeResults: SearchResult[];
    reasoning: ReasoningResult;
  }
): Promise<WritingResult> {
  const system = buildSystemPrompt(personality);
  const user = buildUserPrompt(input);

  return spark.writing.draft({
    prompt: `${system}\n\n${user}`,
    tone: personality.tone,
  });
}

```

src/lib/bud/actions/storeInteraction.ts

```

import type { BudSparkPort } from "../../adapters/sparkPort";

/**
 * Stores both sides of the exchange as episodic memory through Spark.
 * Bud holds no memory of its own – this is the only place a turn is
 * persisted, and it goes straight into Spark.
 */
export function storeInteraction(
  spark: BudSparkPort,
  input: {
    sessionId: string;
    userId: string;
  }
) {

```

```

        userMessage: string;
        budMessage: string;
    }
): void {
    spark.memory.remember({
        kind: "episodic",
        content: `User: ${input.userMessage}`,
        sessionId: input.sessionId,
        userId: input.userId,
        tags: ["conversation", "user"],
    });

    spark.memory.remember({
        kind: "episodic",
        content: `Bud: ${input.budMessage}`,
        sessionId: input.sessionId,
        userId: input.userId,
        tags: ["conversation", "bud"],
    });
}

```

src/lib/bud/prompts/systemPrompt.ts

```

import type { BudPersonality } from "../personality";

/**
 * Pure string assembly from static personality configuration. No
 * decision-making happens here – this is a template, not a model.
 */
export function buildSystemPrompt(personality: BudPersonality): string {
    return [
        `You are ${personality.name}. ${personality.description}`,
        ...personality.voice.map((line) => `- ${line}`),
    ].join("\n");
}

```

src/lib/bud/prompts/userPrompt.ts

```

import type { ReasoningResult } from "../../spark/core/reasoning/interface";
import type { MemoryRecord } from "../../spark/memory/interface";
import type { SearchResult } from "../../spark/intelligence/search/interface";

```

```

/**
 * Assembles the prompt handed to Spark.writing.draft(). All of the
 * "thinking" was already done by Spark (reasoning, search, memory) -
 * this just formats it into text for the writing step.
 */
export function buildUserPrompt(input: {
  message: string;
  memoryRecords: MemoryRecord[];
  knowledgeResults: SearchResult[];
  reasoning: ReasoningResult;
}): string {
  const { message, memoryRecords, knowledgeResults, reasoning } = input;

  const sections = [`User said: "${message}"`, `Spark's analysis: ${reasoning.ans

  if (memoryRecords.length) {
    sections.push(
      `Relevant memory:\n${memoryRecords.map((m) => `- ${m.content}`).join("\n")}
    );
  }

  if (knowledgeResults.length) {
    sections.push(
      `Relevant knowledge:\n${knowledgeResults
        .map((k) => `- ${k.title}: ${k.snippet}`)
        .join("\n")}`
    );
  }

  sections.push("Reply to the user directly, in one short response.");

  return sections.join("\n\n");
}

```